МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ



Лабораторная работа №1

по дисциплине «Технологии разработки программного обеспечения»

на тему

«Разработка класса для нестандартного типа данных на Java»

Группа: АММ26

Факультет: АВТФ

Студент: Арещенко Е.А.

Преподаватель: Токарев В.Г.

Новосибирск 2026

Оглавление

1. Задание на лабораторную работу 3
2. Исходные данные 3
3. Ход работы 3
3.1. Внутреннее представление данных 3
3.2. Конструкторы и деструктор 4
3.3. Доступ к элементам и их изменение 5
3.4. Изменение размерности матрицы 6
3.5. Операции над матрицами 7
3.5.1. Сложение, вычитание, умножение на число 7
3.5.2. Умножение матриц 8
3.5.3. Транспонирование и возведение в степень 8
3.5.4. Определитель 9
3.5.5. Обратная матрица 9
3.5.6. Ранг и след матрицы 10
3.6. Ввод/вывод в текстовый и двоичный поток 10
3.7. Переопределение методов класса Object 11
3.8. Обработка ошибок 12
3.9. Пример работы разработанного ПО 12
Заключение 16
Приложение А 18

1. Задание на лабораторную работу

Разработать класс для представления нестандартного типа данных (см. *cprog* 9.1. Машинная арифметика произвольной точности). Реализовать внутреннее представление данных, конструкторы, деструктор, методы ввода/вывода (сохранения в текстовый и двоичный поток), изменения содержимого отдельных элементов, операции, свойственные данному типу (например, сложение, умножение матриц, вычисление длины и угла вектора).

2. Исходные данные

Согласно варианту задания на лабораторную работу имеем:

- Тип данных: матрица переменной размерности;
- Внутреннее представление: динамический массив указателей на строки матрицы (линейные динамические массивы);
- Язык программирования: *Java* (JDK 21);
- Тип элементов матрицы: вещественный (*double*).

3. Ход работы

Для решения задачи был разработан класс *Matrix*, реализующий матрицу переменной размерности, и класс *Main*, демонстрирующий работу всех методов. Полный исходный код приведён в Приложении А.

3.1. Внутреннее представление данных

Класс *Matrix* содержит следующие поля:

- *private double[][] rows* – динамический массив ссылок (указателей) на строки матрицы; каждая строка – отдельный линейный динамический массив *double[]*;
- *private int nRows* – количество строк;
- *private int nCols* – количество столбцов;
- *private static final double EPS* – точность сравнения вещественных чисел (10^{-10}).

В языке *Java* все массивы являются объектами и доступны только по ссылке, поэтому выражение *double[][]* представляет собой именно массив указателей на строки, а не сплошной блок памяти, как двумерный массив в C. Каждая строка размещается в памяти независимо, что позволяет переставлять, добавлять и удалять строки без копирования остальных данных: поле *rows* хранит ссылки на массивы-строки, а каждый элемент *rows[i]* указывает на массив из *nCols* значений i-й строки (листинг 1).

```
private static final double EPS = 1e-10;

private double[][] rows;
private int nRows;
private int nCols;
```

Листинг 1. Поля класса Matrix

Все поля объявлены с модификатором *private*, доступ к ним осуществляется только через методы класса (инкапсуляция). Это гарантирует, что размеры матрицы всегда соответствуют фактическому содержимому массива.

3.2. Конструкторы и деструктор

В классе реализованы следующие конструкторы:

- *Matrix()* – пустая матрица размером 0×0;
- *Matrix(int nRows, int nCols)* – нулевая матрица заданного размера;
- *Matrix(double[][] data)* – матрица из двумерного массива (данные копируются);
- *Matrix(Matrix other)* – конструктор копирования (глубокое копирование);
- *static Matrix identity(int n)* – фабричный метод создания единичной матрицы.

Основной конструктор выделяет память под массив ссылок, а затем под каждую строку отдельно. Конструктор копирования создаёт новые массивы строк и копирует в них значения методом *System.arraycopy*, поэтому исходная матрица и копия не разделяют общих данных (листинг 2).

```
public Matrix(int nRows, int nCols) {
    if (nRows < 0 || nCols < 0) {
        throw new IllegalArgumentException("Размеры матрицы не могут быть
отрицательными");
    }
    this.nRows = nRows;
    this.nCols = nCols;
    this.rows = new double[nRows][];
    for (int i = 0; i < nRows; i++) {
        rows[i] = new double[nCols];
    }
}

public Matrix(Matrix other) {
    this(other.nRows, other.nCols);
    for (int i = 0; i < nRows; i++) {
        System.arraycopy(other.rows[i], 0, rows[i], 0, nCols);
    }
}
```

Листинг 2. Основной конструктор и конструктор копирования

В языке *Java* отсутствуют деструкторы: освобождением памяти занимается сборщик мусора (*Garbage Collector*), который удаляет объекты, на которые не осталось ссылок. Роль деструктора выполняет метод *close()* интерфейса *AutoCloseable*: он обнуляет ссылки на строки матрицы, делая их доступными для сборщика мусора. Метод может вызываться явно или автоматически при использовании конструкции *try-with-resources* (листинг 3).

```
public void close() {
    if (rows != null) {
        for (int i = 0; i < rows.length; i++) {
            rows[i] = null;
        }
    }
    rows = new double[0][];
    nRows = 0;
    nCols = 0;
}
```

Листинг 3. Метод close() – аналог деструктора

3.3. Доступ к элементам и их изменение

Для чтения и изменения содержимого матрицы реализованы методы:

- *get(int i, int j), set(int i, int j, double value)* – чтение и запись элемента;
- *getRow(int i), setRow(int i, double[] row)* – чтение (копии) и замена строки;
- *swapRows(int i, int k)* – перестановка строк;

- *getRows(), getCols()* – получение размеров матрицы.

Перед каждым обращением выполняется проверка индексов; при выходе за границы генерируется исключение *IndexOutOfBoundsException* с поясняющим сообщением. Метод *swapRows* демонстрирует преимущество выбранного представления: переставляются только ссылки на строки, а сами данные не копируются (листинг 4).

```
public void set(int i, int j, double value) {
    checkIndex(i, j);
    rows[i][j] = value;
}

public void swapRows(int i, int k) {
    checkRow(i);
    checkRow(k);
    double[] tmp = rows[i];
    rows[i] = rows[k];
    rows[k] = tmp;
}
```

Листинг 4. Изменение элемента и перестановка строк

3.4. Изменение размерности матрицы

Поскольку размер массива в *Java* фиксируется при создании, изменение размерности выполняется по принципу динамического массива: создаётся новый массив нужного размера, в него переносятся ссылки (или значения), после чего поле *rows* переключается на новый массив. Старый массив удаляется сборщиком мусора. Реализованы методы:

- *insertRow(int pos, double[] row), addRow(double[] row)* – вставка строки;
- *removeRow(int pos)* – удаление строки;
- *addColumn(double value), removeColumn(int pos)* – добавление и удаление столбца;
- *resize(int newRows, int newCols)* – изменение размеров с сохранением элементов.

При вставке и удалении строки копируются только ссылки на строки, поэтому сложность операции составляет O(n) по числу строк и не зависит от числа столбцов (листинг 5).

```
public void insertRow(int pos, double[] row) {
    if (pos < 0 || pos > nRows) {
        throw new IndexOutOfBoundsException("Неверная позиция строки: " +
pos);
    }
    if (nRows == 0 && nCols == 0) {
        nCols = row.length;
    }
    if (row.length != nCols) {
        throw new IllegalArgumentException("Длина строки должна быть " +
nCols);
    }
    double[][] newRows = new double[nRows + 1][];
    System.arraycopy(rows, 0, newRows, 0, pos);
    newRows[pos] = row.clone();
    System.arraycopy(rows, pos, newRows, pos + 1, nRows - pos);
    rows = newRows;
    nRows++;
}

public void removeRow(int pos) {
    checkRow(pos);
    double[][] newRows = new double[nRows - 1][];
    System.arraycopy(rows, 0, newRows, 0, pos);
    System.arraycopy(rows, pos + 1, newRows, pos, nRows - pos - 1);
    rows = newRows;
    nRows--;
    if (nRows == 0) {
        nCols = 0;
    }
}
```

Листинг 5. Вставка и удаление строки

3.5. Операции над матрицами

Все арифметические операции возвращают новый объект *Matrix* и не изменяют исходные матрицы. Перед выполнением операции проверяется совместимость размеров.

3.5.1. Сложение, вычитание, умножение на число

Операции *add*, *subtract* и *multiply(double k)* выполняются поэлементно и определены для матриц одинакового размера (листинг 6).

```
public Matrix add(Matrix other) {
    checkSameSize(other);
    Matrix res = new Matrix(nRows, nCols);
    for (int i = 0; i < nRows; i++) {
        for (int j = 0; j < nCols; j++) {
            res.rows[i][j] = rows[i][j] + other.rows[i][j];
        }
    }
    return res;
}
```

Листинг 6. Сложение матриц

3.5.2. Умножение матриц

Произведение матриц A (m×n) и B (n×p) определено при совпадении числа столбцов первой матрицы с числом строк второй. Элемент результата вычисляется по формуле $c_{ij} = \Sigma\ a_{ik} \cdot b_{kj}$. Порядок циклов i–k–j выбран так, чтобы внутренний цикл проходил по строкам последовательно, что эффективнее использует кэш процессора (листинг 7).

```
public Matrix multiply(Matrix other) {
    if (nCols != other.nRows) {
        throw new IllegalArgumentException("Нельзя умножить матрицы " +
sizeStr() + " и " + other.sizeStr());
    }
    Matrix res = new Matrix(nRows, other.nCols);
    for (int i = 0; i < nRows; i++) {
        for (int k = 0; k < nCols; k++) {
            double a = rows[i][k];
            for (int j = 0; j < other.nCols; j++) {
                res.rows[i][j] += a * other.rows[k][j];
            }
        }
    }
    return res;
}
```

Листинг 7. Умножение матриц

3.5.3. Транспонирование и возведение в степень

Метод *transpose()* возвращает матрицу, в которой строки и столбцы поменяны местами. Метод *power(int p)* реализует быстрое возведение квадратной матрицы в степень: показатель раскладывается по степеням двойки, благодаря чему требуется O(log p) умножений вместо p−1 (листинг 8).

```
public Matrix power(int p) {
    checkSquare();
    if (p < 0) {
        throw new IllegalArgumentException("Степень должна быть
неотрицательной");
    }
    Matrix result = identity(nRows);
    Matrix base = new Matrix(this);
    while (p > 0) {
        if ((p & 1) == 1) {
            result = result.multiply(base);
        }
        base = base.multiply(base);
        p >>= 1;
    }
    return result;
}
```

Листинг 8. Быстрое возведение в степень

3.5.4. Определитель

Определитель вычисляется методом Гаусса: матрица приводится к верхнетреугольному виду, после чего определитель равен произведению диагональных элементов. При каждой перестановке строк знак определителя меняется. Для уменьшения погрешности вычислений используется выбор главного элемента – в каждом столбце ведущей выбирается строка с максимальным по модулю элементом (метод *findPivot*). Расчёт ведётся на копии матрицы, исходная матрица не изменяется (листинг 9).

```
public double determinant() {
    checkSquare();
    Matrix m = new Matrix(this);
    double det = 1;
    for (int c = 0; c < nRows; c++) {
        int pivot = m.findPivot(c, c);
        if (Math.abs(m.rows[pivot][c]) < EPS) {
            return 0;
        }
        if (pivot != c) {
            m.swapRows(pivot, c);
            det = -det;
        }
        det *= m.rows[c][c];
        for (int i = c + 1; i < nRows; i++) {
            double f = m.rows[i][c] / m.rows[c][c];
            for (int j = c; j < nCols; j++) {
                m.rows[i][j] -= f * m.rows[c][j];
            }
        }
    }
    return det;
}
```

Листинг 9. Вычисление определителя

3.5.5. Обратная матрица

Обратная матрица находится методом Гаусса–Жордана: к исходной матрице приписывается единичная, и над обеими выполняются одинаковые элементарные преобразования до тех пор, пока исходная не станет единичной. При этом приписанная матрица превращается в обратную. Если на очередном шаге ведущий элемент близок к нулю, матрица вырожденная и генерируется исключение *ArithmeticException* (листинг 10).

```
public Matrix inverse() {
    checkSquare();
    int n = nRows;
    Matrix a = new Matrix(this);
    Matrix inv = identity(n);
    for (int c = 0; c < n; c++) {
        int pivot = a.findPivot(c, c);
        if (Math.abs(a.rows[pivot][c]) < EPS) {
            throw new ArithmeticException("Матрица вырожденная, обратной не
существует");
        }
        a.swapRows(pivot, c);
        inv.swapRows(pivot, c);
        double d = a.rows[c][c];
        for (int j = 0; j < n; j++) {
            a.rows[c][j] /= d;
            inv.rows[c][j] /= d;
        }
        for (int i = 0; i < n; i++) {
            if (i != c) {
                double f = a.rows[i][c];
                for (int j = 0; j < n; j++) {
                    a.rows[i][j] -= f * a.rows[c][j];
                    inv.rows[i][j] -= f * inv.rows[c][j];
                }
            }
        }
    }
    return inv;
}
```

Листинг 10. Вычисление обратной матрицы

3.5.6. Ранг и след матрицы

Метод *rank()* приводит копию матрицы к ступенчатому виду и подсчитывает количество ненулевых строк. Метод *trace()* возвращает сумму элементов главной диагонали квадратной матрицы.

3.6. Ввод/вывод в текстовый и двоичный поток

Сохранение и загрузка реализованы на уровне потоков, что позволяет работать как с файлами, так и с любыми другими источниками данных:

- *writeText(Writer)* / *readText(Reader)* – текстовый поток: в первой строке размеры матрицы, далее элементы построчно через пробел;
- *writeBinary(OutputStream)* / *readBinary(InputStream)* – двоичный поток: два целых числа (*int*, по 4 байта) – размеры, затем элементы (*double*, по 8 байт) построчно;

- *saveText*, *loadText*, *saveBinary*, *loadBinary* – методы-обёртки для работы с файлами по имени.

Для двоичного потока используются классы *DataOutputStream* и *DataInputStream*, записывающие примитивные типы в машинно-независимом формате. Файлы открываются в конструкции *try-with-resources*, что гарантирует их закрытие даже при возникновении ошибки (листинг 11).

```
public void writeBinary(OutputStream out) throws IOException {
    DataOutputStream dos = new DataOutputStream(out);
    dos.writeInt(nRows);
    dos.writeInt(nCols);
    for (int i = 0; i < nRows; i++) {
        for (int j = 0; j < nCols; j++) {
            dos.writeDouble(rows[i][j]);
        }
    }
    dos.flush();
}

public static Matrix readBinary(InputStream in) throws IOException {
    DataInputStream dis = new DataInputStream(in);
    int r = dis.readInt();
    int c = dis.readInt();
    Matrix m = new Matrix(r, c);
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            m.rows[i][j] = dis.readDouble();
        }
    }
    return m;
}
```

Листинг 11. Запись и чтение матрицы в двоичном потоке

3.7. Переопределение методов класса Object

В классе переопределены стандартные методы, унаследованные от *Object*:

- *toString()* – форматированное представление матрицы для вывода на экран;
- *equals(Object o)* – сравнение матриц по содержимому с точностью *EPS*;
- *hashCode()* – согласован с *equals* в соответствии с контрактом *Java*.

3.8. Обработка ошибок

Некорректные действия с матрицей приводят к генерации исключений с понятными сообщениями:

- *IllegalArgumentException* – отрицательные размеры, несовместимые размеры матриц при сложении или умножении, строка неверной длины;
- *IndexOutOfBoundsException* – выход индекса строки или столбца за границы;
- *ArithmeticException* – операция над неквадратной матрицей, обращение вырожденной матрицы.

3.9. Пример работы разработанного ПО

В классе *Main* выполняется демонстрация всех возможностей класса *Matrix*: создание матриц, арифметические операции, вычисление определителя, ранга, следа и обратной матрицы, изменение элементов и размерности, сохранение и загрузка из текстового и двоичного файлов, обработка ошибок и освобождение ресурсов. Результаты приведены на рисунках 1–4.

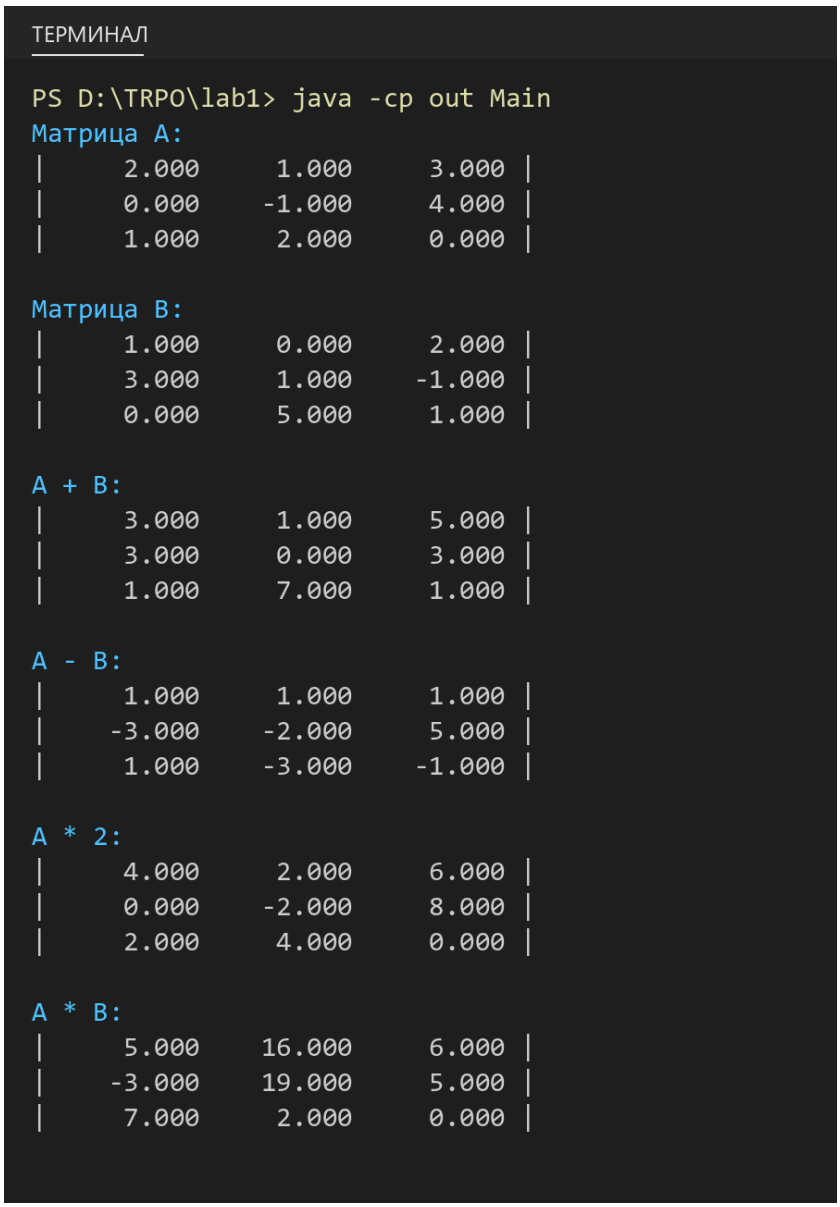


Рисунок 1. Исходные матрицы и арифметические операции

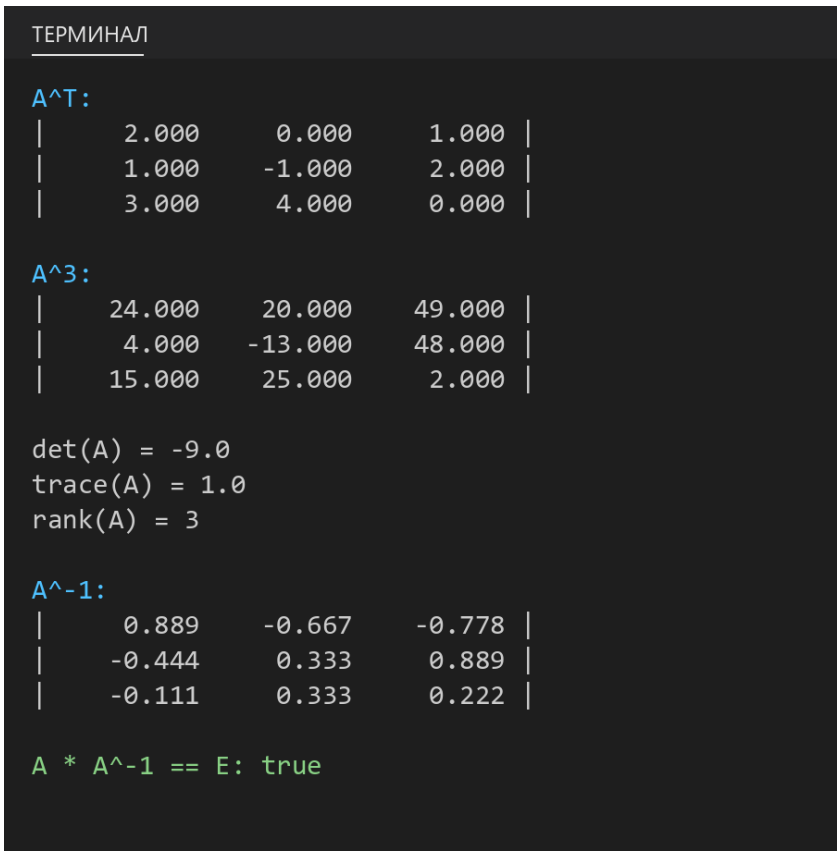


Рисунок 2. Транспонирование, степень, определитель, след, ранг и обратная матрица

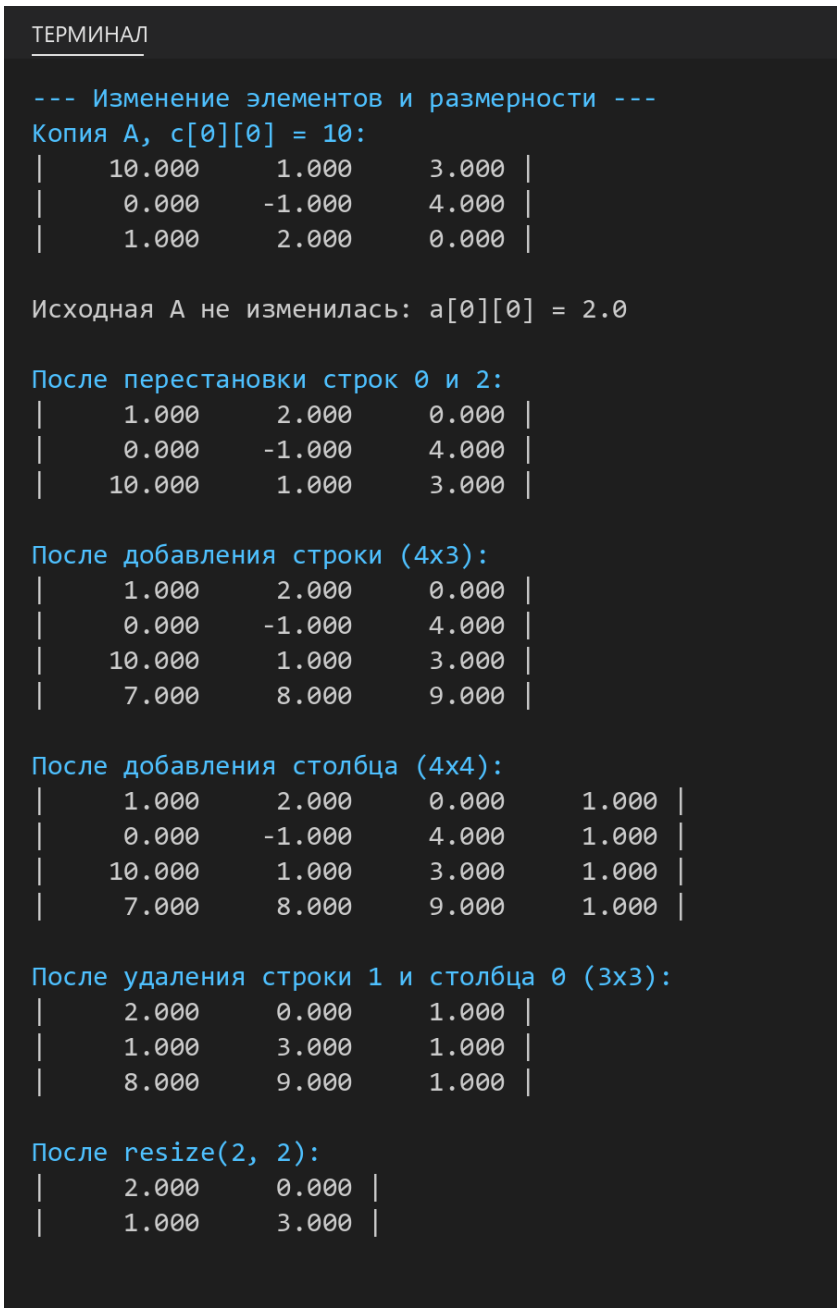


Рисунок 3. Изменение элементов и размерности матрицы

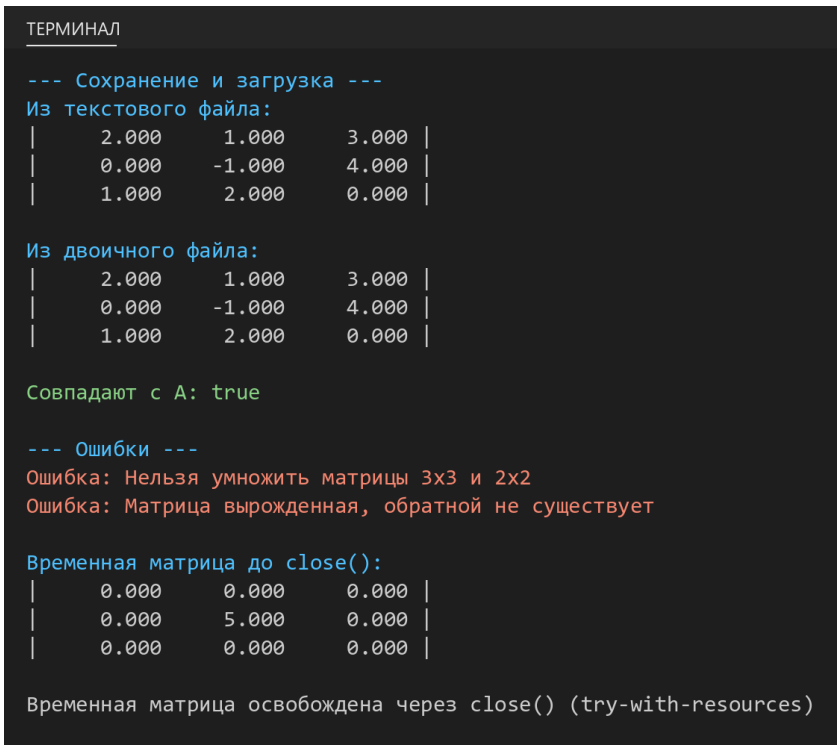


Рисунок 4. Сохранение и загрузка, обработка ошибок, освобождение ресурсов

Корректность вычислений подтверждается проверкой: определитель матрицы A, вычисленный вручную разложением по первой строке, равен $2\cdot(0-8) - 1\cdot(0-4) + 3\cdot(0+1) = -9$, что совпадает с результатом программы; произведение $A\cdot A^{-1}$ равно единичной матрице; матрицы, загруженные из текстового и двоичного файлов, совпадают с исходной.

Заключение

В ходе выполнения лабораторной работы был разработан и протестирован класс *Matrix* для представления нестандартного типа данных «матрица переменной размерности». Внутреннее представление реализовано в виде динамического массива указателей на строки, что позволяет эффективно переставлять, добавлять и удалять строки без копирования данных.

Реализованы конструкторы (в том числе копирования), аналог деструктора на основе интерфейса *AutoCloseable*, методы доступа и изменения отдельных элементов, изменения размерности, сохранения и

загрузки в текстовом и двоичном потоках, а также операции, свойственные матрицам: сложение, вычитание, умножение на число и на матрицу, транспонирование, возведение в степень, вычисление определителя, ранга, следа и обратной матрицы.

Было установлено, что в языке *Java* управление памятью берёт на себя сборщик мусора, поэтому явный деструктор не требуется, а все объекты и массивы доступны по ссылкам, что естественным образом соответствует представлению матрицы в виде массива указателей на строки.

Приложение А

Исходный код класса Matrix (файл Matrix.java)

```
import java.io.*;
import java.util.Locale;
import java.util.Scanner;


public class Matrix implements AutoCloseable {
    private static final double EPS = 1e-10;


    private double[][] rows;
    private int nRows;
    private int nCols;


    public Matrix() {
        this(0, 0);
    }


    public Matrix(int nRows, int nCols) {
        if (nRows < 0 || nCols < 0) {
            throw new IllegalArgumentException("Размеры матрицы не могут быть
отрицательными");
        }
        this.nRows = nRows;
        this.nCols = nCols;
        this.rows = new double[nRows][];
        for (int i = 0; i < nRows; i++) {
            rows[i] = new double[nCols];
        }
    }


    public Matrix(double[][] data) {
        this(data.length, data.length == 0 ? 0 : data[0].length);
        for (int i = 0; i < nRows; i++) {
            if (data[i].length != nCols) {
                throw new IllegalArgumentException("Строки массива имеют разную
длину");
            }
            System.arraycopy(data[i], 0, rows[i], 0, nCols);
        }
    }


    public Matrix(Matrix other) {
        this(other.nRows, other.nCols);
        for (int i = 0; i < nRows; i++) {
            System.arraycopy(other.rows[i], 0, rows[i], 0, nCols);
        }
    }


    public static Matrix identity(int n) {
        Matrix m = new Matrix(n, n);
        for (int i = 0; i < n; i++) {
            m.rows[i][i] = 1;
        }
        return m;
    }


    @Override
    public void close() {
        if (rows != null) {
            for (int i = 0; i < rows.length; i++) {
                rows[i] = null;
            }
        }
```

```
        rows = new double[0][];
        nRows = 0;
        nCols = 0;
    }

    public int getRows() {
        return nRows;
    }

    public int getCols() {
        return nCols;
    }

    public double get(int i, int j) {
        checkIndex(i, j);
        return rows[i][j];
    }

    public void set(int i, int j, double value) {
        checkIndex(i, j);
        rows[i][j] = value;
    }

    public double[] getRow(int i) {
        checkRow(i);
        return rows[i].clone();
    }

    public void setRow(int i, double[] row) {
        checkRow(i);
        if (row.length != nCols) {
            throw new IllegalArgumentException("Длина строки должна быть " + nCols);
        }
        rows[i] = row.clone();
    }

    public void swapRows(int i, int k) {
        checkRow(i);
        checkRow(k);
        double[] tmp = rows[i];
        rows[i] = rows[k];
        rows[k] = tmp;
    }

    public void insertRow(int pos, double[] row) {
        if (pos < 0 || pos > nRows) {
            throw new IndexOutOfBoundsException("Неверная позиция строки: " + pos);
        }
        if (nRows == 0 && nCols == 0) {
            nCols = row.length;
        }
        if (row.length != nCols) {
            throw new IllegalArgumentException("Длина строки должна быть " + nCols);
        }
        double[][] newRows = new double[nRows + 1][];
        System.arraycopy(rows, 0, newRows, 0, pos);
        newRows[pos] = row.clone();
        System.arraycopy(rows, pos, newRows, pos + 1, nRows - pos);
        rows = newRows;
        nRows++;
    }

    public void addRow(double[] row) {
        insertRow(nRows, row);
    }
```

```
    public void removeRow(int pos) {
        checkRow(pos);
        double[][] newRows = new double[nRows - 1][];
        System.arraycopy(rows, 0, newRows, 0, pos);
        System.arraycopy(rows, pos + 1, newRows, pos, nRows - pos - 1);
        rows = newRows;
        nRows--;
        if (nRows == 0) {
            nCols = 0;
        }
    }


    public void addColumn(double value) {
        for (int i = 0; i < nRows; i++) {
            double[] r = new double[nCols + 1];
            System.arraycopy(rows[i], 0, r, 0, nCols);
            r[nCols] = value;
            rows[i] = r;
        }
        nCols++;
    }


    public void removeColumn(int pos) {
        if (pos < 0 || pos >= nCols) {
            throw new IndexOutOfBoundsException("Неверный индекс столбца: " + pos);
        }
        for (int i = 0; i < nRows; i++) {
            double[] r = new double[nCols - 1];
            System.arraycopy(rows[i], 0, r, 0, pos);
            System.arraycopy(rows[i], pos + 1, r, pos, nCols - pos - 1);
            rows[i] = r;
        }
        nCols--;
    }


    public void resize(int newRows, int newCols) {
        if (newRows < 0 || newCols < 0) {
            throw new IllegalArgumentException("Размеры матрицы не могут быть
отрицательными");
        }
        double[][] r = new double[newRows][];
        for (int i = 0; i < newRows; i++) {
            r[i] = new double[newCols];
            if (i < nRows) {
                System.arraycopy(rows[i], 0, r[i], 0, Math.min(nCols, newCols));
            }
        }
        rows = r;
        nRows = newRows;
        nCols = newCols;
    }


    public Matrix add(Matrix other) {
        checkSameSize(other);
        Matrix res = new Matrix(nRows, nCols);
        for (int i = 0; i < nRows; i++) {
            for (int j = 0; j < nCols; j++) {
                res.rows[i][j] = rows[i][j] + other.rows[i][j];
            }
        }
        return res;
    }


    public Matrix subtract(Matrix other) {
        checkSameSize(other);
        Matrix res = new Matrix(nRows, nCols);
```

```
        for (int i = 0; i < nRows; i++) {
            for (int j = 0; j < nCols; j++) {
                res.rows[i][j] = rows[i][j] - other.rows[i][j];
            }
        }
        return res;
    }


    public Matrix multiply(double k) {
        Matrix res = new Matrix(nRows, nCols);
        for (int i = 0; i < nRows; i++) {
            for (int j = 0; j < nCols; j++) {
                res.rows[i][j] = rows[i][j] * k;
            }
        }
        return res;
    }


    public Matrix multiply(Matrix other) {
        if (nCols != other.nRows) {
            throw new IllegalArgumentException("Нельзя умножить матрицы " +
sizeStr() + " и " + other.sizeStr());
        }
        Matrix res = new Matrix(nRows, other.nCols);
        for (int i = 0; i < nRows; i++) {
            for (int k = 0; k < nCols; k++) {
                double a = rows[i][k];
                for (int j = 0; j < other.nCols; j++) {
                    res.rows[i][j] += a * other.rows[k][j];
                }
            }
        }
        return res;
    }


    public Matrix transpose() {
        Matrix res = new Matrix(nCols, nRows);
        for (int i = 0; i < nRows; i++) {
            for (int j = 0; j < nCols; j++) {
                res.rows[j][i] = rows[i][j];
            }
        }
        return res;
    }


    public Matrix power(int p) {
        checkSquare();
        if (p < 0) {
            throw new IllegalArgumentException("Степень должна быть
неотрицательной");
        }
        Matrix result = identity(nRows);
        Matrix base = new Matrix(this);
        while (p > 0) {
            if ((p & 1) == 1) {
                result = result.multiply(base);
            }
            base = base.multiply(base);
            p >>= 1;
        }
        return result;
    }


    public double determinant() {
        checkSquare();
        Matrix m = new Matrix(this);
        double det = 1;
        for (int c = 0; c < nRows; c++) {
```

```
            int pivot = m.findPivot(c, c);
            if (Math.abs(m.rows[pivot][c]) < EPS) {
                return 0;
            }
            if (pivot != c) {
                m.swapRows(pivot, c);
                det = -det;
            }
            det *= m.rows[c][c];
            for (int i = c + 1; i < nRows; i++) {
                double f = m.rows[i][c] / m.rows[c][c];
                for (int j = c; j < nCols; j++) {
                    m.rows[i][j] -= f * m.rows[c][j];
                }
            }
        }
        return det;
    }

    public Matrix inverse() {
        checkSquare();
        int n = nRows;
        Matrix a = new Matrix(this);
        Matrix inv = identity(n);
        for (int c = 0; c < n; c++) {
            int pivot = a.findPivot(c, c);
            if (Math.abs(a.rows[pivot][c]) < EPS) {
                throw new ArithmeticException("Матрица вырожденная, обратной не
существует");
            }
            a.swapRows(pivot, c);
            inv.swapRows(pivot, c);
            double d = a.rows[c][c];
            for (int j = 0; j < n; j++) {
                a.rows[c][j] /= d;
                inv.rows[c][j] /= d;
            }
            for (int i = 0; i < n; i++) {
                if (i != c) {
                    double f = a.rows[i][c];
                    for (int j = 0; j < n; j++) {
                        a.rows[i][j] -= f * a.rows[c][j];
                        inv.rows[i][j] -= f * inv.rows[c][j];
                    }
                }
            }
        }
        return inv;
    }

    public int rank() {
        Matrix m = new Matrix(this);
        int rank = 0;
        for (int c = 0; c < nCols && rank < nRows; c++) {
            int pivot = m.findPivot(rank, c);
            if (Math.abs(m.rows[pivot][c]) < EPS) {
                continue;
            }
            m.swapRows(pivot, rank);
            for (int i = rank + 1; i < nRows; i++) {
                double f = m.rows[i][c] / m.rows[rank][c];
                for (int j = c; j < nCols; j++) {
                    m.rows[i][j] -= f * m.rows[rank][j];
                }
            }
            rank++;
        }
        return rank;
    }
```

```
    public double trace() {
        checkSquare();
        double s = 0;
        for (int i = 0; i < nRows; i++) {
            s += rows[i][i];
        }
        return s;
    }


    public void writeText(Writer out) throws IOException {
        PrintWriter pw = new PrintWriter(out);
        pw.println(nRows + " " + nCols);
        for (int i = 0; i < nRows; i++) {
            StringBuilder sb = new StringBuilder();
            for (int j = 0; j < nCols; j++) {
                if (j > 0) {
                    sb.append(' ');
                }
                sb.append(rows[i][j]);
            }
            pw.println(sb);
        }
        pw.flush();
    }


    public static Matrix readText(Reader in) {
        Scanner sc = new Scanner(in).useLocale(Locale.US);
        int r = sc.nextInt();
        int c = sc.nextInt();
        Matrix m = new Matrix(r, c);
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                m.rows[i][j] = sc.nextDouble();
            }
        }
        return m;
    }


    public void writeBinary(OutputStream out) throws IOException {
        DataOutputStream dos = new DataOutputStream(out);
        dos.writeInt(nRows);
        dos.writeInt(nCols);
        for (int i = 0; i < nRows; i++) {
            for (int j = 0; j < nCols; j++) {
                dos.writeDouble(rows[i][j]);
            }
        }
        dos.flush();
    }


    public static Matrix readBinary(InputStream in) throws IOException {
        DataInputStream dis = new DataInputStream(in);
        int r = dis.readInt();
        int c = dis.readInt();
        Matrix m = new Matrix(r, c);
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                m.rows[i][j] = dis.readDouble();
            }
        }
        return m;
    }


    public void saveText(String fileName) throws IOException {
        try (Writer w = new FileWriter(fileName)) {
            writeText(w);
```

```
        }
    }

    public static Matrix loadText(String fileName) throws IOException {
        try (Reader r = new FileReader(fileName)) {
            return readText(r);
        }
    }

    public void saveBinary(String fileName) throws IOException {
        try (OutputStream os = new BufferedOutputStream(new
FileOutputStream(fileName))) {
            writeBinary(os);
        }
    }

    public static Matrix loadBinary(String fileName) throws IOException {
        try (InputStream is = new BufferedInputStream(new
FileInputStream(fileName))) {
            return readBinary(is);
        }
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < nRows; i++) {
            sb.append("| ");
            for (int j = 0; j < nCols; j++) {
                sb.append(String.format(Locale.US, "%9.3f ", rows[i][j]));
            }
            sb.append("|\n");
        }
        return sb.toString();
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (!(o instanceof Matrix other) || nRows != other.nRows || nCols !=
other.nCols) {
            return false;
        }
        for (int i = 0; i < nRows; i++) {
            for (int j = 0; j < nCols; j++) {
                if (Math.abs(rows[i][j] - other.rows[i][j]) > EPS) {
                    return false;
                }
            }
        }
        return true;
    }

    @Override
    public int hashCode() {
        return 31 * nRows + nCols;
    }

    private int findPivot(int fromRow, int col) {
        int pivot = fromRow;
        for (int i = fromRow + 1; i < nRows; i++) {
            if (Math.abs(rows[i][col]) > Math.abs(rows[pivot][col])) {
                pivot = i;
            }
        }
```

```
        return pivot;
    }


    private String sizeStr() {
        return nRows + "x" + nCols;
    }


    private void checkRow(int i) {
        if (i < 0 || i >= nRows) {
            throw new IndexOutOfBoundsException("Неверный индекс строки: " + i);
        }
    }


    private void checkIndex(int i, int j) {
        checkRow(i);
        if (j < 0 || j >= nCols) {
            throw new IndexOutOfBoundsException("Неверный индекс столбца: " + j);
        }
    }


    private void checkSameSize(Matrix other) {
        if (nRows != other.nRows || nCols != other.nCols) {
            throw new IllegalArgumentException("Размеры матриц не совпадают: " +
sizeStr() + " и " + other.sizeStr());
        }
    }


    private void checkSquare() {
        if (nRows != nCols) {
            throw new ArithmeticException("Операция определена только для квадратной
матрицы");
        }
    }
}
```

Исходный код класса Main (файл Main.java)

```
import java.io.IOException;


public class Main {
    public static void main(String[] args) throws IOException {
        Matrix a = new Matrix(new double[][]{
                {2, 1, 3},
                {0, -1, 4},
                {1, 2, 0}
        });
        Matrix b = new Matrix(new double[][]{
                {1, 0, 2},
                {3, 1, -1},
                {0, 5, 1}
        });


        System.out.println("Матрица A:\n" + a);
        System.out.println("Матрица B:\n" + b);


        System.out.println("A + B:\n" + a.add(b));
        System.out.println("A - B:\n" + a.subtract(b));
        System.out.println("A * 2:\n" + a.multiply(2));
        System.out.println("A * B:\n" + a.multiply(b));
        System.out.println("A^T:\n" + a.transpose());
        System.out.println("A^3:\n" + a.power(3));
        System.out.println("det(A) = " + a.determinant());
        System.out.println("trace(A) = " + a.trace());
        System.out.println("rank(A) = " + a.rank());
```

```
        Matrix inv = a.inverse();
        System.out.println("\nA^-1:\n" + inv);
        System.out.println("A * A^-1 == E: " +
a.multiply(inv).equals(Matrix.identity(3)));

        System.out.println("\n--- Изменение элементов и размерности ---");
        Matrix c = new Matrix(a);
        c.set(0, 0, 10);
        System.out.println("Копия A, c[0][0] = 10:\n" + c);
        System.out.println("Исходная A не изменилась: a[0][0] = " + a.get(0, 0));
        c.swapRows(0, 2);
        System.out.println("\nПосле перестановки строк 0 и 2:\n" + c);
        c.addRow(new double[]{7, 8, 9});
        System.out.println("После добавления строки (" + c.getRows() + "x" +
c.getCols() + "):\n" + c);
        c.addColumn(1);
        System.out.println("После добавления столбца (" + c.getRows() + "x" +
c.getCols() + "):\n" + c);
        c.removeRow(1);
        c.removeColumn(0);
        System.out.println("После удаления строки 1 и столбца 0 (" + c.getRows() +
"x" + c.getCols() + "):\n" + c);
        c.resize(2, 2);
        System.out.println("После resize(2, 2):\n" + c);

        System.out.println("--- Сохранение и загрузка ---");
        a.saveText("matrix.txt");
        a.saveBinary("matrix.bin");
        Matrix fromText = Matrix.loadText("matrix.txt");
        Matrix fromBin = Matrix.loadBinary("matrix.bin");
        System.out.println("Из текстового файла:\n" + fromText);
        System.out.println("Из двоичного файла:\n" + fromBin);
        System.out.println("Совпадают с A: " + (a.equals(fromText) &&
a.equals(fromBin)));

        System.out.println("\n--- Ошибки ---");
        try {
            a.multiply(new Matrix(2, 2));
        } catch (IllegalArgumentException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }
        try {
            new Matrix(new double[][]{{1, 2}, {2, 4}}).inverse();
        } catch (ArithmeticException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }

        try (Matrix tmp = new Matrix(3, 3)) {
            tmp.set(1, 1, 5);
            System.out.println("\nВременная матрица до close():\n" + tmp);
        }
        System.out.println("Временная матрица освобождена через close() (try-with-
resources)");
    }
}
```